

Case Study: Building a Document-Based AI Assistant with LangChain and OpenAI

Overview

In July 2025, I successfully designed, built, and deployed a fully functional AI assistant that uses Retrieval-Augmented Generation (RAG) to answer user questions based on personal documents. The assistant integrates LangChain, FAISS, and OpenAI's GPT-4o model, all orchestrated through a FastAPI backend and a Streamlit frontend.

Objective

To create a helpful, context-aware AI tool capable of ingesting personal documents and answering user questions in a conversational, human-like manner. The system needed to:

- Understand document context
- Maintain conversation memory per user
- Stream responses in real time
- Be deployed as a complete working prototype

Tools and Technologies

- **LangChain** for building the RAG pipeline
- **FAISS** for vector-based document retrieval
- **OpenAI GPT-4o** for LLM reasoning and natural response generation
- **FastAPI** to serve the backend and memory logic
- **Streamlit** for the web-based chat interface
- **Pydantic** for request validation
- **Python (asyncio, dotenv, etc.)** for supporting infrastructure

Architecture

1. **Frontend:** A Streamlit UI that collects the user's name, document uploads, and questions.
2. **Backend:** A FastAPI server exposing an `/ask` endpoint, managing:
3. Chat memory per user via `ConversationBufferMemory`
4. LangChain's `ConversationalRetrievalChain`
5. Streaming responses using LangChain callbacks
6. **Core Components:**
7. FAISS index for document storage and retrieval
8. GPT-4o as the LLM with `streaming=True`
9. Prompt templates encouraging natural, opinionated, and helpful responses

Key Features

- **Memory Per User:** Each user's conversation is tracked via `chat_id` ensuring personalized and context-aware responses.
- **Streaming Architecture:** Implemented a token-wise response stream with a custom `StreamHandler` class to simulate ChatGPT-like behavior.
- **Modular Design:** Clean separation of logic between `main.py`, `api.py`, and the Streamlit frontend.
- **Friendly AI Personality:** Prompt engineered for warmth, clarity, and human-like tone.

Challenges & Solutions

1. Streaming Limitations in Streamlit

Streamlit does not support true async streaming. I implemented a partial workaround using `st.empty()` and LangChain's `on_llm_new_token`, though full streaming requires backend UI frameworks like React.

2. Async/Await Errors

Initially, I misused `asyncio.run()` on a non-coroutine. The issue was resolved by avoiding async endpoints when unnecessary or restructuring the code to support proper async handling.

3. Noisy API Responses

LangChain's `.invoke()` returned structured responses including metadata. I switched to `.run()` for simpler, ChatGPT-like string answers, resulting in a cleaner user experience.

4. Deployment

Managing environment variables, FAISS index loading, and ensuring CORS compatibility were critical for deploying the backend.

Lessons Learned

- Modular architecture is crucial for scalability and debugging.
- Memory management with LangChain is powerful for multi-user interactions.
- Prompt design significantly shapes the AI's tone and usefulness.
- Streaming requires careful thought when working outside native async UI frameworks.
- Debugging LangChain chains, especially with callbacks and memory, requires patience and clear logs.

Outcome

- Deployed a working AI assistant accessible via browser
- Integrated LLM, document retrieval, and memory handling into a cohesive system
- Learned real-world LangChain, OpenAI, FastAPI, and Streamlit deployment skills

Next Steps

- Improve streaming experience with a React or WebSocket-based UI
- Add document upload-to-index pipeline
- Expand memory to long-term vector storage across sessions

GitHub Repository

https://github.com/havryleshko/llm_assistant

This project marks a major milestone in my journey as an ML/AI engineer and a document-based RAG system designer. It taught me not only technical integration but also how to shape a product that genuinely helps users interact with their own data intelligently.